

MiRust Language Specification (v1.0)

This document defines the formal language specification for **MiRust (v1.0)**, a statically and strongly typed procedural programming language designed for semester-long compiler implementation projects.

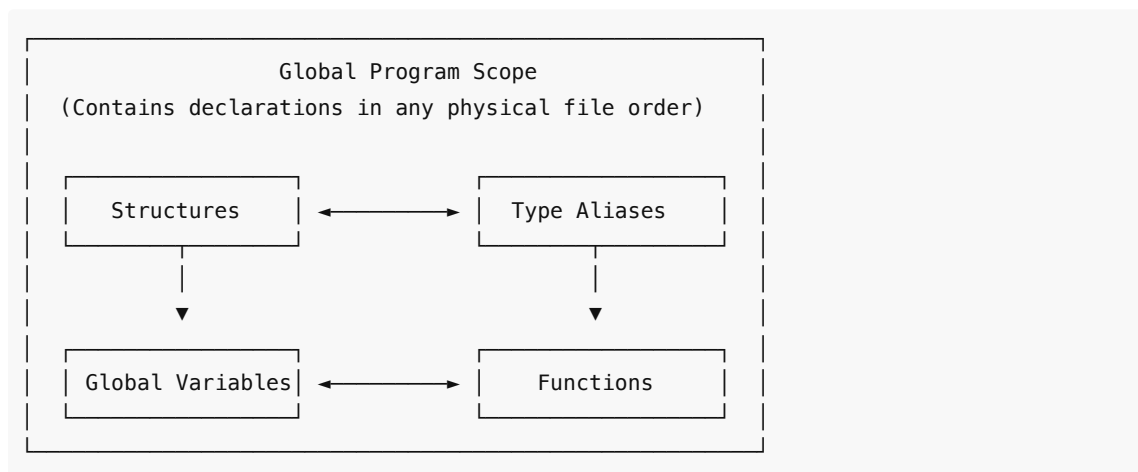
MiRust blends a safe subset of Rust's design paradigms—such as immutability by default, block-structured lexical scoping, type aliasing, explicit box-based heap allocation, and copy/move ownership semantics—with classical C-style control flow constructs (such as single-statement conditional blocks and switch statements with implicit fall-through and unified lexical scopes).

1. Introduction and Program Layout

A MiRust source file represents **a single compilation unit**. All global definitions are visible throughout the entire program. Unlike strictly ordered single-pass compilers, MiRust permits global declarations to **appear in any physical order** at the top level of the file.

A valid MiRust program must contain a minimum of **one global declaration** (specifically, the mandatory `main` function definition to serve as the entry point). The program consists of a collection of the following global declarations:

- **Structure Definitions**
- **Type Declarations (Aliases)**
- **Global Variable Declarations**
- **Function Definitions**



All structure definitions, type aliases, global variables, and functions reside in the same global namespace and are recursively and mutually visible. Forward referencing is supported natively; a global entity may reference another global entity defined physically later in the source file.

2. Lexical Grammar

2.1. Identifiers and Keywords

- **Identifiers:** Must **begin with a letter or an underscore**, followed by **zero or more characters that can be letters, digits, or underscores**.
 - Identifiers are strictly **case-sensitive** (e.g., `myVar` and `myvar` are distinct identifiers).

- The term "**letter**" refers exclusively to any of the twenty-six standard uppercase English alphabetic characters from A through Z, or any of the twenty-six standard lowercase English alphabetic characters from a through z.
- **Keywords:** The following words are reserved and cannot be used as identifiers: `let`, `mut`, `type`, `struct`, `fn`, `void`, `i32`, `f32`, `bool`, `string`, `Box`, `true`, `false`, `if`, `else`, `switch`, `case`, `default`, `while`, `for`, `in`, `break`, `continue`, `return`, `nil`.

2.2. Literals

- **Integer Literals:** Consist of a sequence of one or more decimal digits representing a **32-bit integer**.
- **Float Literals:** Consist of an integer part (one or more decimal digits) followed by a fractional part and/or an exponent part. The fractional part and the exponent part are both optional, but **at least one of them must be present**.
 - **Fractional Part:** Consists of a decimal point followed by one or more decimal digits.
 - **Exponent Part:** Starts with the letter `e` or `E`, followed by an optional sign (`+` or `-`) and one or more decimal digits.
 - *Valid Examples:* `1.0e1`, `3.1415e-2`, `0.003E+5` (both present); `1.0`, `0.0001` (fractional part only); `1e5`, `12E-3` (exponent part only).
 - *Invalid Examples:* `1.` (trailing decimal point with no fractional digits and no exponent); `1.0e` (missing digits in exponent).
- **Boolean Literals:** The literal terms `true` or `false`.
- **String Literals:** A sequence of characters enclosed in double quotes. String literals cannot contain raw newline or raw tab characters directly. Supported escape sequences within string literals are a backslash followed by the character `n` for newlines, `t` for tabs, `another backslash` for a literal backslash, or a **double quote for a literal double quote**.
- **Box Literals:** The literal term `nil` represents a null or empty Box pointer. `nil` is polymorphically compatible with any `Box<T>` type.
- **Array and Struct Literals:** Array and Struct literals will be described in Section 3.

2.3. Comments

- **Single-line Comments:** Begin with **two consecutive forward slash characters** and extend to the end of the current line.
- **Multi-line Comments:** Begin with **a forward slash and an asterisk**, and end with **an asterisk and a forward slash**. Nesting of multi-line comments is not supported.

3. Type System

MiRust features a strict type system. Aside from a single, explicit implicit conversion rule, all types must match exactly in assignments, function calls, and operations.

3.1. Primitive Types

- `i32`: Signed 32-bit two's complement integer.
- `f32`: 32-bit IEEE 754 single-precision float.
- `bool`: Logical values `true` and `false`.
- `string`: Immutable, read-only sequence of UTF-8 characters.
- `void`: Represents the empty tuple type. Used exclusively as the return type for functions that do not return a value.

3.2. Complex and Composite Types

- **Static Arrays:** Fixed-size contiguous blocks of memory denoted by a type and a positive integer size enclosed in brackets, separated by a semicolon (e.g., `[i32; 5]`).
 - *Array Literals:* An array literal consists of a **non-empty**, comma-separated list of expressions enclosed in square brackets (e.g., `[1, 2, 3]`).
 - *Type Compatibility:* Two array types are compatible if and only if they have the **exact same element type and identical size**. Assigning an array literal with a differing size or element type (e.g., assigning `[1]` or `[1, 2, 3]` to a variable of type `[i32; 2]`) is a compile-time **Type Mismatch**.
 - *Indexing:* Written using bracket notation `expr1[expr2]` , where `expr1` is an expression evaluating to a static array type `[T; N]` and `expr2` is an index expression that must evaluate or coerce to the `i32` type. Elements are 0-indexed from `0` to `N - 1` .
- **Structures (Structs):** User-defined composite types containing named, typed fields.
 - *Type Equivalence:* Structures implement **Nominal Equivalence**. Two structures are considered strictly distinct types if they have different names, even if their field definitions are structurally identical.
 - *Declaration:* Defined with the `struct` keyword, followed by an identifier, followed by a brace-enclosed list of fields, where each field consists of an identifier and a type separated by a colon, and fields are separated by commas.
 - *Minimum Fields:* A structure definition must contain **at least one field**. Empty structures (structures with zero fields) are syntactically invalid.
 - *Struct Literals:* A struct literal is constructed by specifying the structure's type identifier, followed by an opening curly brace `{` , a comma-separated list of field initializers, and a closing curly brace `}` .
 - Each field initializer is written as `field_name: expression` .
 - All fields defined in the structure declaration must be initialized exactly once.
 - Field initializers may appear in **any order** (unordered initialization) regardless of their physical declaration order in the struct definition.
 - *Example:* `Point { y: 2.0, x: 1.0 }` is a valid literal for `struct Point { x: f32, y: f32 }` .
 - *Field Access:* Written using a postfix dot operator (`.`) followed by the field identifier (e.g., `expr.field_name`).
 - The base expression (`expr`) must evaluate to a structure type. If `expr` evaluates to a boxed pointer of a structure type (`Box<NamedType>`), it must be explicitly dereferenced first using the unary dereference operator (e.g., `(*boxed_expr).field_name`) because implicit pointer dereferencing is not supported.
 - Accessing a field identifier that does not exist in the structure definition triggers an **Undeclaration Error**.
- **Boxed Pointer Types (`Box<T>`):** Dynamic heap-allocated pointers containing a value of type `T` .
 - *Allocation:* Instantiated using the intrinsic function `Box_new` passing an expression as an argument. If the expression has type `T` , the instantiation expression evaluates to type `Box<T>` .
 - *Dereferencing:* The underlying value is accessed using a unary prefix asterisk operator.

3.3. Type Declarations (Aliases)

Type declarations introduce a new name for an existing type. They are declared globally using the `type` keyword, followed by the new alias name, an equals sign, the existing target type, and a terminating semicolon:

```
type CoreInt = i32;
type PointArray = [Point; 10];
```

- Type aliases are fully transparent, implementing structural equivalence (unlike structures which implement nominal equivalence). An alias name can be used interchangeably with its underlying type.

3.4. Recursive and Mutually Recursive Reference Rules

Because all global structures and type definitions are mutually visible, recursive references are resolved using the following static rules:

- **Type Aliases:** Cyclic or recursive type alias definitions are strictly forbidden. Under structural equivalence, cycles like `type A = B; type B = A;` or `type T = Box<T>;` represent infinite expansions and must trigger a compile-time **Type Resolution Error**.
- **Structure Definitions:** A structure definition cannot contain a direct field of its own type, nor a field of another structure type that mutually references it, because this requires infinite storage size.
 - *Invalid:* `struct Node { next: Node }` (Infinite physical layout).
 - *Valid:* A structure may contain recursive or mutually recursive fields *only if* the recursive reference path is broken by at least one pointer indirection (`Box<T>`).
 - *Valid Example:*

```
struct Node {
    data: i32,
    next: Box<Node> // Allowed because Box size is statically
known
}
```

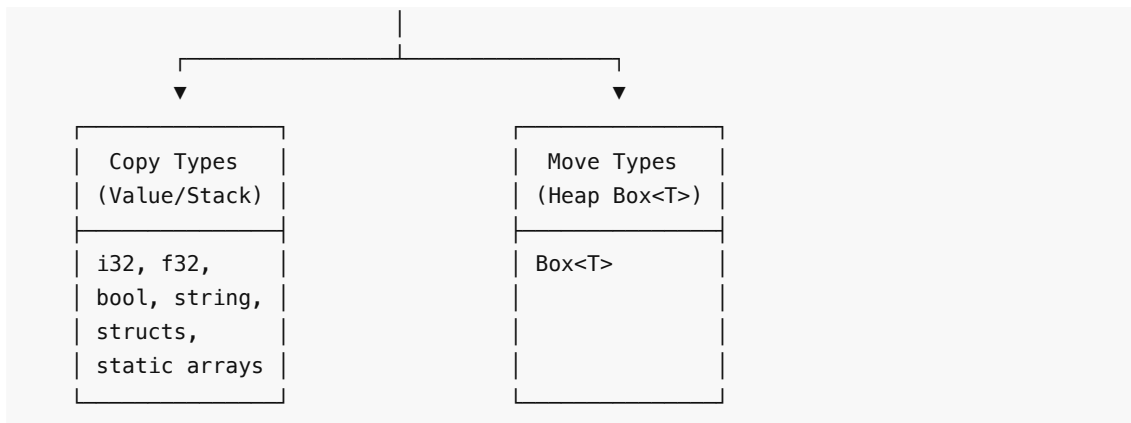
3.5. Implicit Type Coercion

- An `i32` value is promoted to `f32` in binary operations if and only if the other operand is an `f32`.
- An `i32` expression is promoted to `f32` when assigned to an `f32` target (including variable assignments, function argument passing, and return statements).
- The `nil` literal is implicitly polymorphic and is treated as the target `Box<T>` type when used in assignments, array/struct initialization, function parameters, and return statements.
- In binary equality operations (`==` , `!=`), `nil` can be compared with any operand of type `Box<T>` or another `nil` literal.
- All other mismatched types trigger static **Type Mismatch Errors**.

4. Copy and Move Semantics

To manage system resources and references safely without a runtime garbage collector, MiRust categorizes all types into either **Copy** or **Move** categories. MiRust has neither a Garbage Collector (GC) nor explicit `free` memory semantics. When a heap-allocated `Box` goes out of scope or its ownership is lost, its memory is intentionally leaked.

MiRust Types



4.1. Value Classifications

- **Copy Types:** `i32` , `f32` , `bool` , `string` , all `struct` types, and all static arrays. Values of Copy types use value semantics: assignments, function arguments, returns, and struct/array initializations create independent copies of the value.
- **Move Types:** `Box<T>` (heap-allocated pointers). Values of Move types use single-ownership semantics: when assigned or passed, ownership of the underlying heap resource is transferred to the destination.

4.2. Ownership Assignment and Move Rules for `Box<T>`

When a value of a **Copy Type** is assigned or passed, the value is duplicated. The source variable remains initialized and fully usable.

When a value of a **Move Type** (`Box<T>`) is transitioned via any of the following operations, ownership of the heap pointer is transferred from the source to the target:

1. **Variable Initialization:** Initializing a new variable with the value of an existing `Box<T>` variable.
2. **Assignment:** Assigning an existing `Box<T>` variable to another variable.
3. **Function Arguments:** Passing a `Box<T>` variable as an argument to a function parameter.
4. **Function Returns:** Returning a `Box<T>` value from a function.
5. **Composite Construction:** Placing an existing `Box<T>` variable inside an array literal or a struct instantiation.

*[!WARNING] If ownership of a `Box<T>` variable is (or **might be** via branching) transferred, the source variable—whether mutable or immutable—transitions to an **Uninitialized (Moved-Out)** state. Any subsequent access to a moved-out `Box<T>` variable before reinitialization is a compile-time **Use-After-Move Error**.*

*[!NOTE] **Static vs. Runtime Checking Boundary:***

- **Compile-time checking (Static Analysis):** The `UseAfterMoveOrUninitialized` error is strictly checked at the **whole-variable identifier level** (`Id`). The static checker tracks whether declared variables are initialized before use or moved out.
- **Runtime checking (Execution):** Individual struct fields or array elements are not statically tracked for moved/uninitialized states. Any attempt to dereference an invalid or `nil` pointer at runtime will trigger a runtime error.

4.3. Reinitialization

A mutable variable (declared with `let mut`) of type `Box<T>` that has been moved out can be restored to the **Initialized** state by assigning a new, valid value to it:

```
let mut b = Box_new(10);
let c = b; // b is now moved-out and uninitialized
// println_i32(*b); // COMPILE ERROR: Use-after-move!

b = Box_new(20); // b is now reinitialized and valid
println_i32(*b); // Valid: outputs 20
```

5. Variables, Constants, Scopes, and Shadowing

5.1. Immutability by Default

All variables declared with `let` are strictly immutable. The `mut` modifier must be specified to allow reassignments:

```
let x = 5;           // Immutable
// x = 6;           // COMPILE ERROR: Mutability Violation

let mut y = 5;       // Mutable
y = 6;               // Valid
```

5.2. Declaration Syntax

Variables must be declared one at a time. Multi-variable declarations (such as declaring multiple identifiers separated by commas in a single statement) are syntactically invalid.

- Explicitly typed with initializer:** The keyword `let`, an optional `mut` keyword, the variable name, a colon, the type, an equals sign, the initialization expression, and a terminating semicolon.
- Explicitly typed, uninitialized:** The keyword `let`, the keyword `mut`, the variable name, a colon, the type, and a terminating semicolon. This is strictly forbidden for immutable variables; an immutable variable *must* have an initializer.
- Implicitly typed with initializer:** The keyword `let`, an optional `mut` keyword, the variable name, an equals sign, the initialization expression, and a terminating semicolon. The type of the variable is statically inferred from the type of the initialization expression.

5.3. Global Variables

Global variables are declared outside of functions. They:

- Must have an explicit type annotation.
- Must be initialized with a compile-time constant expression, which can be a literal, a basic binary arithmetic operation on constant literals, or a parenthesized constant expression.
- Can be mutable (`let mut`) or immutable (`let`).

5.4. Lexical Scope and Shadowing

- **Scoping:** Variables are strictly block-scoped. They are visible from their point of declaration until the closing curly brace of the block in which they are declared.
- **Shadowing:** Local variables in nested blocks can shadow variables of the exact same name from outer scopes. Redefining a variable of the same name within the *same* immediate local scope

block, however, is a compile-time **Redeclaration Error**.

6. Statements and Control Flow

Statements are the primary units of execution in a MiRust program. In MiRust, variable declarations are formally classified as statements (specifically, declaration statements) and can appear anywhere a statement is permitted, including inside blocks.

6.1. Variable Declaration Statements

A variable declaration (as specified in Section 5) is a statement. It introduces and optionally initializes a variable in the local block scope.

6.2. Expression Statements

An expression statement is just an expression followed by a semicolon.

6.3. Blocks (`{ ... }`)

A block consists of an opening curly brace `{`, followed by zero or more statements, followed by a closing curly brace `}`.

- A block introduces a nested lexical scope. Local variables declared inside a block are discarded at the end of the block.
- Statements within a block execute sequentially.
- Unlike Rust, blocks in MiRust are not expressions and do not produce a value; they are structures containing statements.

6.4. Conditional Statements (`if - else`)

A conditional statement consists of the keyword `if`, followed by a condition expression enclosed in parentheses, followed by a statement, and optionally followed by the keyword `else` and a second statement.

- The control condition within the parentheses must evaluate to the `bool` type.
- The conditional statements can be single statements or compound block statements enclosed in curly braces.
- **Dangling `else` Resolution:** An `else` clause is always associated with the nearest preceding open `if` statement in the same block.

```
if (x > 0)
    if (y > 0)
        print_string("Both positive");
    else
        print_string("x positive, y non-positive"); // Binds to the second 'if'
```

6.5. Iteration Statements (`while` and `for`)

A while loop consists of the keyword `while`, followed by a condition expression enclosed in parentheses, followed by a statement.

- The control condition within the parentheses must evaluate to the `bool` type.

A for loop consists of the keyword `for`, followed by a parenthesized clause containing a loop variable identifier, the keyword `in`, a start expression, two consecutive dots representing a range, and an end expression, followed by a statement.

- The start and end expressions must evaluate to `i32` values and are evaluated exactly once before the loop begins.
- The loop iterates over the half-open interval from the start value up to, but not including, the end value.
- The loop variable is an implicit, immutable variable of type `i32` defined within a nested loop scope. Attempting to assign to or mutate this loop variable is a compile-time **Mutability Violation**. If a variable of the same name is declared inside the loop body, it shadows the implicit loop variable for the remainder of its block scope, which is permitted and does not trigger a **Redeclaration Error**.

6.6. Switch Statements

The `switch` statement provides multi-way branching based on an integer or boolean value.

A switch statement consists of the keyword `switch`, followed by a parenthesized control expression, and a body block enclosed in curly braces. The body block contains zero or more case arms, followed optionally by a single default arm. Each case arm begins with the keyword `case`, followed by a constant expression, a colon, and zero or more statements. The optional default arm begins with the keyword `default`, followed by a colon and zero or more statements.

- **Condition:** The control expression must evaluate to `i32` or `bool`.
- **Implicit Fall-Through:** Control flows sequentially into the subsequent case or default statement list unless interrupted by a `break`, `continue`, or `return` statement.
- **Unified Scope:** The entire body of the switch statement, spanning all case arms and the default arm, shares a single, unified lexical scope.

*[!IMPORTANT] Because of unified scoping in `switch`, declaring a variable inside one case (e.g., `case 1: let mut x = 10;`) makes `x` visible in all other cases. However, to ensure type safety under fall-through and direct jumps, any variable declared within a `switch` is considered uninitialized at the start of the `switch`. Jumping over its declaration line (e.g., entering `case 2` directly when `x` was declared in `case 1`) and reading it without an intervening initialization triggers a compile-time **Use of Uninitialized Variable Error**.*

```
switch (value) {
  case 1:
    let mut x = 42; // x is declared in switch-scope
    print_i32(x);
    break;
  case 2:
    // x is visible here because of unified scope, but uninitialized if we jump
    // straight to case 2!
    x = 99; // Valid: Reinitializes x
    print_i32(x);
    break;
  default:
    // print_i32(x); // COMPILE ERROR: x is uninitialized here if switch jumped
    // directly to default
}
```

6.7. Loop Control Statements

- `break;` : Terminates execution of the innermost loop (`while` or `for`) or exits the innermost `switch` block. Using `break` outside of an enclosing loop or switch statement is a compile-time **Invalid Control Flow Error**.
- `continue;` : Skips the remainder of the current iteration of the innermost loop. Using `continue` outside of an enclosing loop block is a compile-time **Invalid Control Flow Error**.

6.8. Return Statements

The `return` statement immediately terminates execution of the current function, returning control (and optionally a value) back to the caller.

- **With Value:** `return expression;`
 - The expression is evaluated, and its static type must match the declared return type of the enclosing function (subject to implicit `i32` -to- `f32` coercion).
 - If the enclosing function's return type is `void` , returning a value triggers a compile-time **Type Mismatch Error**.
 - **Without Value:** `return;`
 - This form is only valid in functions where the declared return type is `void` (or omitted). Returning without a value in a non- `void` function triggers a compile-time **Type Mismatch Error**.
 - An implicit `return;` is automatically executed at the end of any function returning `void` . Static checking only validates the type consistency of `return` statements whenever they occur; static path reachability analysis (e.g., verifying that all branches of `if / while / switch` contain a `return`) is not required.
-

7. Functions and the Standard Library

7.1. Function Declarations and Calls

Functions are defined using the `fn` keyword, followed by an identifier, a parenthesized parameter list, an optional return arrow and return type, and a block of statements enclosed in curly braces.

- The parameter list is a list of parameters separated by commas (`,`). It contains a minimum of **zero parameters**. Each parameter is written as `parameter_name: type` .
- Function calls are written by specifying the function identifier followed by a parenthesized argument list. The argument list is a list of expressions separated by commas (`,`), containing a minimum of **zero arguments**.
- If the return type arrow and type are omitted, the return type defaults to `void` .
- Parameter names must be unique within the parameter list.
- Parameters are treated as local, immutable variables within the function body. Mutating a parameter directly is a compile-time error unless it is shadowed by a mutable local declaration.
- Parameters of **Move Types** transfer ownership into the function when called.

7.2. Entry Point

Every executable program must contain exactly one `main` function. This function acts as the execution entry point.

- The `main` function must accept zero parameters and must have a `void` return type:

```
fn main() -> void {
    // Execution begins here
}
```

- The absence of a valid `main` function must trigger a compile-time **No Entry Error**.

7.3. Mutual Visibility

Functions reside in a mutually visible global scope. A function can call any other function defined in the same file, regardless of their relative physical order of definition. Recursive and mutually recursive function calls are fully supported.

7.4. Standard Library Built-Ins

The following runtime output functions and intrinsic functions are predefined in the global environment:

- `print_i32(val: i32)` , `println_i32(val: i32)`
- `print_f32(val: f32)` , `println_f32(val: f32)`
- `print_bool(val: bool)` , `println_bool(val: bool)`
- `print_string(val: string)` , `println_string(val: string)`
- `println()` (Prints a newline character)
- `Box_new(val: T) -> Box<T>` (Intrinsic compiler function that allocates a value of type `T` on the heap)

[!IMPORTANT] Scope and Reservation Rules for Built-Ins:

- **Global Scope Reservation:** All predefined built-in names (`print_*` , `println_*` , `println`) reside in the global symbol table. Attempting to declare a user-defined global function, global variable, struct, or type alias with any of these names triggers a compile-time **Redeclaration Error** (`Redeclared`).
- **Intrinsic `Box_new` :** `Box_new` is a reserved polymorphic intrinsic operation for heap allocation.
- **Local Block Shadowing:** In accordance with standard lexical scoping (§5.4), an inner local block may declare a local variable that shares a name with a built-in function (e.g., `let print_i32 = 10;`). Within that block, the identifier resolves to the local variable, and attempting to invoke it as a function will result in an `Undeclared(Function, ...)` error.

8. Expressions, Operators, and Precedence

8.1. Definition of Expressions

An expression is a syntactic construct that evaluates to a single value of a statically determinable type. Expressions can be grouped into:

- **Literals:** Integer, float, boolean, and string constant values.
- **Variable References:** Evaluating the value stored inside local or global variables.
- **Operations:** Binary and unary operators applied to sub-expressions.
- **Function Calls:** Evaluating the return value of a function.
- **Compound Construction:** Creating instance values of structs (`Point { x: 1.0, y: 2.0 }`) and arrays (`[1, 2, 3]`).
- **Accessors:** Field projection (`obj.x`) and array subscripting (`arr[i]`).

- **Indirection Operators:** Boxing values (`Box_new(expr)`) and dereferencing pointers (`*my_box`).

Expressions are classified into **LValues** (representing writeable/addressable memory locations) and **RValues** (temporary values).

8.2. LValues

An expression is a valid **LValue** if it is:

1. A variable name (local or global).
2. An array index access expression: `arr[i]` .
3. A struct field access expression: `obj.field` .
4. A dereferenced box: `*my_box` .

An LValue is **mutable** if its base identifier was declared with `mut` , or if it is a path derived from a mutable variable.

8.3. Operator Precedence Table

The following table lists operators from highest precedence (1) to lowest precedence (9).

Precedence	Operator	Description	Associativity
1	<code>()</code> , <code>[]</code> , <code>.</code> , <code>f(...)</code>	Function Call, Array Index, Field Access	Left-to-Right
2	<code>-</code> , <code>!</code> , <code>*</code>	Unary Minus, Logical NOT, Dereference	Right-to-Left
3	<code>*</code> , <code>/</code> , <code>%</code>	Multiplicative Operators	Left-to-Right
4	<code>+</code> , <code>-</code>	Additive Operators	Left-to-Right
5	<code><</code> , <code><=</code> , <code>></code> , <code>>=</code>	Relational Comparisons	Non-associative
6	<code>==</code> , <code>!=</code>	Equality Comparisons	Non-associative
7	<code>&&</code>	Logical AND	Left-to-Right
8	<code> </code>	Logical OR	Left-to-Right
9	<code>=</code>	Assignment	Right-to-Left

*[!NOTE] Relational and Equality operators are strictly **non-associative**. An expression like `a < b < c` is syntactically invalid and must trigger a compile-time **Syntax Error**. Parentheses must be explicitly used: `(a < b) && (b < c)` .*

Logical operators (`&&` and `||`) enforce **short-circuit evaluation**. The right-hand operand is only evaluated if the left-hand operand does not fully determine the result of the expression.

The return value of the assignment operator is the value of the right-hand side.

8.4. Valid Types for Operators

To enforce type safety and memory safety, operators are strictly limited to the following operands. Any violation triggers a compile-time **Type Mismatch Error**:

- **Unary Minus (`-`):** The operand must be strictly of type `i32` or `f32` . The result type matches the operand type.

- **Logical NOT (!)**: The operand must be strictly of type `bool` . The result type is `bool` .
- **Dereference (*)**: The operand must evaluate to a boxed pointer `Box<T>` . The result type is the underlying type `T` .
- **Binary Logical Operators (&& , ||)**: Both operands must be strictly of type `bool` . The result type is `bool` .
- **Binary Arithmetic Operators (+ , - , * , /)**: Both operands must be `i32` or `f32` (subject to implicit `i32` to `f32` promotion if one is `f32`). The result type matches the (promoted) operand type.
- **Modulo (%)**: Both operands must be strictly `i32` . The result type is `i32` .
- **Relational Operators (< , <= , > , >=)**: Both operands must be `i32` or `f32` (subject to promotion). The result type is `bool` .
- **Equality Operators (== , !=)**: Allowed **only** for the following types:
 - **Primitive Types**: `i32` , `f32` , `bool` , `string` (operands must match in type, subject to `i32` to `f32` promotion). For `string` , comparison is content-based (value equality).
 - **Pointer Types (Box<T>)**:
 - **Comparison Semantics**: Compares **pointer addresses / references** (reference equality), returning `true` if and only if both pointers point to the exact same heap memory location or both are `nil` . It does **not** compare the underlying values on the heap. (To compare the underlying boxed values, explicit dereferencing must be used: `*b1 == *b2`).
 - **Type Compatibility**: `Box<T1>` compared with `Box<T2>` requires `T1` and `T2` to be compatible matching types under the MiRust type system (type aliases are transparently resolved; struct types follow **Nominal Equivalence** and must have the same struct name; array types must have matching sizes and element types).
 - **Comparison with nil** : `Box<T>` can be compared with `nil` , and `nil` can be compared with `nil` .

9. Static Validation & Error Classification

Compilers must run lexical, syntactic, and semantic analyses to validate code correctness prior to code generation.

9.1. Lexical Errors

Triggered during tokenization when the scanner encounters characters or sequences that do not conform to the lexical grammar rules of the language.

- *Examples*: Unrecognized characters (e.g., `@` , `#`), an unterminated string literal, or an invalid escape sequence inside a string literal.

9.2. Syntax Errors

Triggered during parsing if the token stream violates grammar rules, program layout rules, or operator associativity constraints.

- *Example*: `a < b < c` (Violation of non-associative operators).

9.3. Type Mismatch Errors

Triggered when expected types do not align with actual types (subject to the `i32` to `f32` promotion exception). This category includes, but is not limited to:

- **Conditional Parameters:** The condition guard of an `if` statement or a `while` loop must evaluate strictly to the `bool` type.
- **Loop Range Bounds:** The start and end expressions of a `for` loop range must evaluate strictly to the `i32` type.
- **Array Indexing:** The index expression within an array subscript operator (e.g., `arr[index]`) must evaluate or coerce to the `i32` type.
- **Switch Statements:** The control expression of a `switch` statement must evaluate to either `i32` or `bool`, and all constant expressions in its `case` arms must match the type of the control expression.
- **Assignments:** Assigning a value of type `U` to an LValue of type `T` where `U` cannot be coerced to `T`.
- **Function Arguments:** Passing an argument expression whose type does not match the corresponding declared parameter type in a function call.
- **Return Values:** A `return` expression whose type does not match the declared return type of the enclosing function.
- **Operator Operands:** Applying a binary or unary operator to operands of incompatible types (e.g., adding a `string` to an `i32`).
- **Variable Declarations:** Providing an initial value whose type does not match the declared type, or declaring an immutable variable without providing an initial value.

9.4. Mutability Violations

Triggered when an operation attempts to reassign or mutate an immutable LValue.

- *Example:* Reassigning a variable declared without `mut`, mutating a function parameter, or mutating the loop index variable inside a `for` loop body.

9.5. Undeclaration Errors

Triggered when an identifier (representing a variable, constant, function, structure, field or type alias) is referenced in an expression or statement without having been declared beforehand or globally.

- *Example:* `x = 99;` or `print_i32(x);` when `x` has not been declared in any accessible scope, or calling a non-existent function.

9.6. Redeclaration Errors

Triggered when an identifier is declared multiple times within the same scope.

- *Examples:* Two global variables of the same name, duplicate fields in a struct definition, duplicate local variables within the same block level, or declaring a user-defined global entity (function, variable, struct, alias) with the name of a built-in standard library function (e.g., `fn print_i32()`).

9.7. Use-After-Move and Uninitialized Errors

Triggered when a variable is accessed but is either uninitialized or has had its ownership moved away.

- *Example:*

```
let x = Box_new(5);
let y = x; // x moved
let z = *x; // COMPILE ERROR: Use-after-move!
```

9.8. No Entry Errors

Triggered when the compiler is unable to find a valid program entry point.

- *Example:* The input file does not define a `fn main() -> void` function, or the `main` function contains parameters or a non-`void` return type.

9.9. Type Resolution Errors

Triggered when the compiler cannot infer a concrete type (e.g., initializing with `let x = nil;` in an implicitly typed declaration) or when there are invalid, cyclic, or infinite-sized type relationships in structure or alias definitions (e.g., `type A = B; type B = A;` or `struct S { value: S }`).

9.10. Invalid Control Flow Errors

Triggered when a loop control statement is used outside of its allowed context.

- *Examples:* Using `continue` outside of an enclosing loop block, or using `break` outside of an enclosing loop or switch statement.

9.11. Struct Initialization Errors

Triggered during struct instantiation if a struct field is left **un-initialized** or is **re-initialized** multiple times in the struct literal.

9.12. Global Initialization Errors

Triggered when a global variable is **not initialized** or is initialized by an expression that is **not a compile-time constant expression**.

10. Code Examples

10.1. Iterative Fibonacci using Type Aliases and Single-Statement Loops

```
type CoreInt = i32;

fn main() -> void {
    let mut a: CoreInt = 0;
    let mut b: CoreInt = 1;
    let mut i: CoreInt = 0;

    while (i < 10) {
        let temp = a + b;
        a = b;
        b = temp;
        i = i + 1;
    }
    println_i32(a);
}
```

10.2. Heap Allocation, Ownership Moves, and Struct Interaction

```

struct Vector3D {
    x: f32,
    y: f32,
    z: f32
}

fn process_vector(v: Box<Vector3D>) -> f32 {
    // v owns the boxed vector now.
    return (*v).x + (*v).y + (*v).z;
}

fn main() -> void {
    let mut my_vec = Vector3D { x: 1.0e0, y: 2.0e0, z: 3.0e0 };
    let boxed_vec = Box_new(my_vec); // my_vec is a Copy type: its value is
    duplicated to the heap; my_vec remains valid!

    println_f32(my_vec.x); // Valid: structs are Copy types, my_vec is NOT moved!

    let sum = process_vector(boxed_vec); // boxed_vec (Box<Vector3D>, Move type) is
    moved into process_vector!

    // println_f32((*boxed_vec).x); // COMPILE ERROR: Use-after-move!
    println_f32(sum);
}

```